

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Cheredla Jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	
--------------	-----	--	--	--------------------------------	------------------------------	--

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. Face Detection and Recognition Using Eigenfaces

Answers:

Objective

Detect multiple faces in a grayscale image and recognize each detected face using the **Eigenfaces/PCA method**, even with moderate lighting variations.

Algorithm

Input: Grayscale image of size

Output: Bounding boxes and recognized names

1. Load the grayscale input image.
2. Normalize the image intensity.
3. Apply histogram equalization to reduce the effect of varying illumination.
4. Detect candidate faces using a **Viola–Jones detector** or sliding-window method.
5. **For every detected face:**
 - Crop the face using its bounding box.
 - Resize it to a fixed size.
 - Normalize its intensity.
6. **Using the training database:**
 - Convert all training faces into vectors.
 - Calculate the mean face.
 - Subtract the mean face from each training image.
 - Apply PCA to obtain eigenfaces.
7. Project each training face onto the eigenface space.
8. Project every detected face onto the same eigenface space.
9. Calculate the Euclidean distance between the detected face and each known face.
10. Select the person with the smallest distance.
11. If the minimum distance is below a predefined threshold, assign that person's name; otherwise label the face Unknown.
12. Draw the bounding box and recognized name on the original image.
13. Repeat for all detected faces.

Pseudocode

Algorithm Eigenface_Face_Recognition

Input:

Grayscale image I of size $M \times N$

Training database $D = \{\text{face images with names}\}$

Output:

Bounding boxes and recognized names

Begin

1. **Preprocess input image:**

```

    I ← NormalizeIntensity(I)
    I ← HistogramEqualization(I)
2. Detect faces:
    FaceBoxes ← ViolaJonesDetector(I)
3. Prepare training database:
    For each training face F in D:
        F ← Resize(F, fixed_width, fixed_height)
        F ← NormalizeIntensity(F)
        Convert F into vector
    End For
4. Compute mean face:
    MeanFace ← Average(all training face vectors)
5. Compute eigenfaces using PCA:
    For each training face F:
        A ← F - MeanFace
    End For
    Eigenfaces ← PCA(A)
6. Project training faces:
    For each training face F:
        FeatureF ← Project(F - MeanFace, Eigenfaces)
    End For
7. For each box B in FaceBoxes:
    Face ← Crop(I, B)
    Face ← Resize(Face, fixed_width, fixed_height)
    Face ← NormalizeIntensity(Face)
    Feature ← Project(Face - MeanFace, Eigenfaces)
    For each known person P:
        Distance[P] ← EuclideanDistance(
            Feature,
            FeatureF[P])
    End For
    BestPerson ← Person with minimum Distance
    If Distance[BestPerson] < RecognitionThreshold:
        Label ← Name(BestPerson)
    Else
        Label ← "Unknown"
    End If
    DrawBoundingBox(I, B)
    DrawLabel(I, B, Label)
8. Return I containing all bounding boxes and labels.
End

```

Handling varying lighting

Histogram equalization and intensity normalization reduce illumination differences. For better performance, adaptive histogram equalization (CLAHE) or illumination-normalization techniques can also be used.

2. Real-Time Road Lane Detection

Objective

Detect and highlight lane lines in every frame of a driving video while handling noise, shadows, curves, and partial occlusion.

Algorithm :

Input: Video sequence from a front-facing camera

Output: Each video frame with detected lanes highlighted

- Capture a video frame.
- Convert the frame to grayscale.
- Apply Gaussian smoothing to remove noise.
- Apply Canny edge detection to identify lane boundaries.
- Define a Region of Interest (ROI) containing the road.
- Apply the Hough Transform to detect lane-line candidates.
- For curved lanes, use polynomial fitting or a perspective-transform-based method.
- Filter out incorrect line segments based on their position and slope.
- Combine the detected lane lines.
- Overlay the lane lines on the original frame.
- Display the processed frame.
- Repeat until the video ends.

Pseudocode

Algorithm RealTime_Lane_Detection

Input:

Driving video V

Output:

Video frames with detected lanes highlighted

Begin

1. Open video V

2. While video contains frames:

Frame $\leftarrow$ CaptureNextFrame(V)

3. Gray $\leftarrow$ ConvertToGrayscale(Frame)

4. Smooth $\leftarrow$ GaussianBlur(Gray)

5. Edges $\leftarrow$ CannyEdgeDetection(Smooth)

6. Define Region of Interest:

ROI $\leftarrow$ SelectRoadRegion(Edges)

7. Detect lane candidates:

Lines $\leftarrow$ HoughTransform(ROI)

8. Filter detected lines:

Remove lines with invalid slope,
position, or length

9. If curved lanes are present:

LaneCurve $\leftarrow$ PolynomialFit(EdgePoints)

10. Combine left and right lane boundaries:

LeftLane $\leftarrow$ EstimateLeftLane(Lines)

RightLane $\leftarrow$ EstimateRightLane(Lines)

11. Create lane overlay:

LaneImage $\leftarrow$ DrawLanes(

```
        LeftLane,  
        RightLane)  
12.  OutputFrame ← Overlay(  
        Frame,  
        LaneImage)  
13.  Display(OutputFrame)  
14. End While  
15. Close video  
End
```

Improving robustness

Shadows: Use color-space processing and adaptive thresholding rather than relying only on grayscale intensity.

Curved roads: Use polynomial curve fitting, sliding-window detection, or perspective transformation instead of only straight-line Hough detection.

Partial occlusion: Track lane positions across consecutive frames and use the previous frame's lane estimate when a lane temporarily disappears.

Real-time performance: Restrict processing to the road ROI and use temporal filtering to stabilize lane estimates.